Declaración de uso de inteligencia artificial en el proyecto final

Análisis de Algoritmos

Nicolas Buitrago Martinez

1092851684

Juan David Martinez Valencia

1095208100

Luisa Fernanda Castañeda Cano

1097040431



Docente: Sergio Augusto Cardona Torres

Universidad del Quindío

Facultad de Ingeniería

Programa Ingeniería de Sistemas y Computación

Armenia

2026

Contenido

1. Propósito de la declaración 3
2. Herramienta utilizada 3
3. Para qué usamos la herramienta 3
4. Prompts utilizados 4
5. Cómo revisamos y validamos las respuestas 6
6. Alcance y responsabilidad 7
7. Referencias bibliográficas 7

1. Propósito de la declaración

El enunciado del proyecto final exige declarar de forma explícita el uso de herramientas de inteligencia artificial generativa como apoyo al desarrollo y aclara que estas pueden servir de soporte, pero no reemplazar el diseño algorítmico ni el análisis formal solicitados en el curso (Universidad del Quindío, 2026). Este documento acompaña el documento técnico y declara qué herramienta usamos, para qué la usamos, cuáles fueron los prompts que le dimos y cómo revisamos y validamos lo que nos entregó.

Conviene distinguir dos usos de la IA en el proyecto. El primero es parte de la solución: los modelos GloVe y all-MiniLM-L6-v2, que el requerimiento 1 pide usar para generar embeddings, están documentados en la sección 4.6 del documento técnico. El segundo, que es el objeto de esta declaración, es el asistente de IA generativa que usamos como apoyo durante el desarrollo.

2. Herramienta utilizada

Tabla 1

Herramienta de IA generativa utilizada

Aspecto	Detalle
Herramienta	Claude, de Anthropic (modelo Opus 5.5)
Forma de uso	Claude Code, en la aplicación de escritorio de Claude, con acceso a la carpeta del proyecto en nuestro computador
Período	Octubre y noviembre de 2026, en la fase final del proyecto
Punto de partida	Nuestro avance del proyecto en Visual Studio Code, el seguimiento del ranking de términos con sus observaciones y el enunciado del proyecto final

3. Para qué usamos la herramienta

Usamos el asistente como apoyo en seis frentes. La Tabla 2 resume cada uno con ejemplos concretos del proyecto; la sección 4 relaciona cada frente con los prompts que lo originaron.

Tabla 2

Usos del asistente de IA en el proyecto

Uso	En qué consistió	Ejemplos en el proyecto
Asesoría	Resolver dudas sobre los algoritmos, su complejidad y la forma de demostrarlos y medirlos	Cómo medir tiempos en la JVM sin sesgos (calentamiento, ejecuciones intercaladas, mediana y rango intercuartílico); cómo evaluar el agrupamiento con la correlación cofenética y la silueta
Comparación	Contrastar alternativas antes de decidir y comparar resultados entre algoritmos	Criterios de enlace del agrupamiento; modelos de embeddings disponibles; servicios de despliegue (Render frente a otras opciones); lectura crítica de las similitudes clásicas frente a las de IA
Investigación	Consultar técnicas, bibliotecas y referencias	Modelos preentrenados para vectorizar los abstracts (FastEmbed, GloVe, MiniLM); jerarquía de memoria y caché para explicar las mediciones; referencias bibliográficas de cada algoritmo
Mejoras	Proponer y escribir mejoras al código y a la interfaz, que luego revisamos y probamos	Rediseño de la interfaz; vista paso a paso de la búsqueda; año de publicación en la tabla del corpus; grupos del dendrograma con título y autores
Actualizaciones	Incorporar las observaciones del docente y mantener coherentes el código y los documentos	Programa de verificación en Python en el repositorio; 30 ejecuciones con tablas numéricas; experimento de memoria caché; integración del ranking de términos al proyecto final
Despliegue	Acompañamiento completo en la publicación de la aplicación	Dockerfile de dos etapas, creación del repositorio en GitHub, configuración del servicio web en Render y verificación de la aplicación desplegada

También lo usamos para revisar la redacción del documento técnico y para generar sus figuras a partir de los archivos de resultados. Las cifras de los documentos salen de nuestras propias ejecuciones de la aplicación.

4. Prompts utilizados

La Tabla 3 presenta los prompts principales en el orden en que los usamos. Están resumidos, sin cambiar lo que pedimos; omitimos los mensajes breves de seguimiento, como «continúa» o «inténtalo de nuevo», y las respuestas a preguntas puntuales del asistente, como la visibilidad del repositorio.

Tabla 3

Prompts dados al asistente

N.°	Prompt (resumen)	Uso	Resultado
1	A partir de nuestro avance del proyecto y de las observaciones del docente al seguimiento (verificador en Python fuera del repositorio, explicación débil de la caché, solo tres ejecuciones y sin tablas numéricas, falta de la declaración de IA y del formato del documento), apóyanos para completar lo pendiente de los requerimientos y revisa el documento en Word.	Asesoría, actualizaciones, mejoras	Revisión del avance frente al enunciado; implementación de lo pendiente; correcciones del seguimiento; documento técnico revisado
2	Despliega la aplicación en Render, pero solo si el proyecto cumple el enunciado que te entregamos.	Despliegue, comparación	Verificación de los cuatro requerimientos antes de desplegar; Dockerfile; prueba local de la imagen
3	Crea un repositorio nuevo en nuestra cuenta de GitHub, súbelo, despliega desde allí y agrega a JuanDaM01 como colaborador.	Despliegue	Repositorio público bibliometria-proyecto-final; servicio web en Render; invitación al colaborador
4	Las correcciones del seguimiento deben quedar en el proyecto desplegado y en el documento nuevo; mejora la apariencia de la interfaz con las skills de diseño impeccable, taste-skill y emilkowalski/skill, y explícanos cómo subir nosotros los cambios.	Actualizaciones, mejoras, investigación	Paquete ranking en la aplicación, pestaña Ranking de términos, interfaz rediseñada; instrucciones para publicar con GitHub Desktop
5	En la tabla del corpus las palabras y los autores se ven pequeños frente al título y no aparece el año de publicación; mejora esa parte.	Mejoras	Extracción del año desde la cita de cada revista; tabla del corpus con año y palabras legibles
6	Integra el capítulo del ranking al documento, analiza de nuevo el enunciado y lo que falta a nivel de código, crea el documento de prompts y una presentación del proyecto.	Asesoría, comparación, mejoras	Revisión del código frente al enunciado; vista paso a paso de la búsqueda; grupos con título y autores; capítulo 7 del documento; esta declaración; presentación
7	Agrégame como autora y construye en la pestaña de similitud una vista que muestre paso a paso cómo se llena la matriz de Levenshtein y la de Needleman-Wunsch.	Mejoras, actualizaciones	Consulta /api/matriz; vista Matriz paso a paso en la pestaña Similitud; métodos de camino óptimo; 3 pruebas JUnit

N.°	Prompt (resumen)	Uso	Resultado
8	Implementa la búsqueda de varios patrones con Rabin-Karp y compárala con KMP y Boyer-Moore; revisa todo el código como un revisor (código sin uso, validación de la API, accesibilidad); actualiza el documento técnico, esta declaración y la presentación, y verifica el proyecto frente al enunciado antes de la entrega.	Mejoras, comparación, actualizaciones, despliegue	Rabin-Karp multipatrón y /api/busqueda-multiple; validación de la API; 9 pruebas JUnit nuevas; informe de revisión del código; documento técnico, declaración y presentación actualizados

5. Cómo revisamos y validamos las respuestas

Antes de incorporar un cambio propuesto por el asistente ejecutamos la aplicación y las pruebas, y contrastamos los resultados con fuentes independientes:

- Pruebas unitarias. El proyecto tiene 52 pruebas que comparan cada algoritmo con resultados conocidos o con una referencia: la distancia de Levenshtein de «kitten» y «sitting», la función de prefijos de «ababaca», los criterios de enlace sobre puntos en una recta, la búsqueda por fuerza bruta en 300 textos aleatorios y el ordenamiento estable de Java.
- Cálculos a mano. Los casos de estudio del documento técnico (artículos 11 y 16 para la similitud y el patrón «analysis» para la búsqueda) se pueden seguir paso a paso y coinciden con lo que calcula la aplicación.
- Verificación independiente. El conteo de términos y el ranking se repitieron con un programa en Python que no reutiliza el código Java, y coincidieron en las 950 posiciones.
- Revisión contra los PDF. El título, los autores, el año y el abstract de cada artículo se compararon con la primera página del PDF correspondiente.
- Revisión de la aplicación desplegada. Después de cada despliegue revisamos en Render que las cuatro pestañas respondieran con los mismos resultados que la versión local.

- Pruebas de las funciones finales. La búsqueda de varios patrones y la matriz paso a paso se probaron con pruebas JUnit que comparan sus resultados con KMP, con la fuerza bruta y con cada algoritmo de similitud; además consultamos la API con parámetros inválidos, que deben responder con un error 400 y un mensaje claro, y revisamos la interfaz en escritorio y en celular.

Esta revisión encontró problemas que se corrigieron antes de la entrega. MiniLM procesa como máximo 256 tokens y 13 abstracts eran más largos, así que se habrían truncado sin aviso; ahora se dividen en oraciones y se promedian sus vectores. En el artículo 13 el encabezado de margen estaba en la misma línea que el título, lo que confundía la extracción, y el artículo 2 aparecía como de 2022 porque ese es el año de su aviso de derechos de autor, aunque pertenece al volumen de 2023.

6. Alcance y responsabilidad

Las decisiones del proyecto las tomamos nosotros: qué algoritmos implementar, qué casos de estudio analizar, cómo interpretar las mediciones, qué mejoras incorporar y dónde desplegar la aplicación. Los algoritmos de los tres requerimientos y los del ranking están implementados de forma explícita, con estructuras básicas de Java y sin funciones de alto nivel que los resuelvan, como exige el enunciado, de modo que su comportamiento se puede seguir y analizar línea por línea.

Asumimos la responsabilidad por todo el contenido entregado, incluido el código que el asistente propuso o escribió y que nosotros revisamos, ejecutamos y aceptamos. Usar el asistente nos permitió dedicar más tiempo a entender y comparar los algoritmos, y también nos mostró la importancia de verificar cada resultado con fuentes independientes antes de darlo por correcto.

7. Referencias bibliográficas

Anthropic. (2026). *Claude* (versión Opus 5.5) [Modelo de lenguaje de gran tamaño]. https://claude.ai

Universidad del Quindío. (2026). *Análisis de algoritmos en el contexto de la bibliometría: Descripción final del proyecto* [Enunciado de la actividad]. Programa de Ingeniería de Sistemas y Computación.

VIGILADA MINEDUCACIÓN



PROGRAMA DE INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

Tel: (57) 6 735 9300 Ext 355

Carrera 15 Calle 12 Norte

Armenia, Quindío – Colombia

ingesis@uniquindio.edu.co